

Trzeci zestaw zadań (Kompresja Macierzy)

Robert Mesek

December 12, 2024

1 Kompresja bitmapy RGB

1.1 Zadanie

- Proszę wybrać kolorową bitmapę
- Proszę zamienić bitmapę na 3 macierze Red, Green, Blue (wartości z przedziału 0-255)
- Proszę napisać rekurencyjną kompresję macierzy z wykorzystaniem częściowego SVD dla wybranych parametrów σ = najmniejsza wartość osobliwa (wyrzucamy mniejsze) i b = maksymalny rank (liczba wartości osobliwych)
- Proszę zaimplementować rysowawcz skompresowanej macierzy
- Proszę zaimplementować rysowacz skompresowanej bitmapy

```
[1]: from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.image import imread

np.random.seed(0)
```

```
[9]: def partial_svd(matrix: NDArray, k: int) -> tuple[NDArray, NDArray, NDArray]:
    """
    Parameters
    A : ndarray
        Matrix to decompose of a floating point numeric dtype.
    k : int
        Number of singular values and singular vectors to compute.
        Must satisfy 1 <= k <= kmax, where kmax=min(M, N).

    Returns
    u : ndarray, shape=(M, k)
        Unitary matrix having left singular vectors as columns.
    s : ndarray, shape=(k,)
        The singular values (in descending order).
    vt : ndarray, shape=(k, N)
        Unitary matrix having right singular vectors as rows.
    """
    u, s, vt = np.linalg.svd(matrix, full_matrices=False)
```

```
return u[:, :k], s[:k], vt[:k, :]
```

```
[10]: @dataclass
class CompressedNode:
    # Hierarchical SVD representation
    u: Optional[NDArray[np.float32]] = None
    s: Optional[NDArray[np.float32]] = None
    vt: Optional[NDArray[np.float32]] = None

    # Children nodes
    # tl / tr
    # ---/---
    # bl / br
    tl: Optional["CompressedNode"] = None # top-left
    tr: Optional["CompressedNode"] = None # top-right
    bl: Optional["CompressedNode"] = None # bottom-left
    br: Optional["CompressedNode"] = None # bottom-right

    # Helpers
    @property
    def is_leaf(self) -> bool:
        return (
            (self.tl is None)
            and (self.tr is None)
            and (self.bl is None)
            and (self.br is None)
        )

    @property
    def matrix(self) -> NDArray[np.float32]:
        if self.is_leaf:
            assert self.u is not None, "Matrix must be defined"
            assert self.s is not None, "Matrix must be defined"
            assert self.vt is not None, "Matrix must be defined"

            return np.clip(self.u @ np.diag(self.s) @ self.vt, 0.0, 1.0)
        else:
            assert self.tl is not None, "Top-left child must be defined"
            assert self.tr is not None, "Top-right child must be defined"
            assert self.bl is not None, "Bottom-left child must be defined"
            assert self.br is not None, "Bottom-right child must be defined"

            tl = self.tl.matrix
            tr = self.tr.matrix
            bl = self.bl.matrix
            br = self.br.matrix
```

```
return np.block([[tl, tr], [bl, br]])
```

```
[11]: def create_tree(
    matrix: NDArray[np.float32], max_block_size: int, sv_threshold: float
) -> CompressedNode:
    m, n = matrix.shape
    u, s, vt = partial_svd(matrix, max_block_size)

    # s[-1] = smallest singular value
    if s[-1] < sv_threshold or min(m, n) <= 2 * max_block_size + 1:
        return CompressedNode(u, s, vt)

    node = CompressedNode()
    m2, n2 = m // 2, n // 2
    node.tl = create_tree(matrix[:m2, :n2], max_block_size, sv_threshold)
    node.tr = create_tree(matrix[:m2, n2:], max_block_size, sv_threshold)
    node.bl = create_tree(matrix[m2:, :n2], max_block_size, sv_threshold)
    node.br = create_tree(matrix[m2:, n2:], max_block_size, sv_threshold)
    return node
```

```
[12]: def get_sv_threshold(
    singular_values: NDArray[np.float32], target_svd_components: int
) -> float:
    assert (
        0 <= target_svd_components <= len(singular_values)
    ), f"`target_svd_components` range is (0, {len(singular_values)})"

    if target_svd_components == 1:
        return float("inf")

    elif target_svd_components == 0 or target_svd_components == len(
        singular_values
    ):
        return float("-inf")

    else:
        return (
            singular_values[target_svd_components - 2]
            + singular_values[target_svd_components - 1]
        ) / 2
```

1.2 Reprezentacja macierzy kompresji

```
[13]: def fill_matrix_image(node, matrix_image, i, j, m, n):
    if node.is_leaf:
        x_0, y_0 = i, j
        x_1, y_1 = i + m, j + n
```

```

    # Black lines thickness
    x_size = node.s.shape[0]
    y_size = node.s.shape[0]

    matrix_image[x_0 : x_0 + x_size, y_0:y_1] = 0.0
    matrix_image[x_0:x_1, y_0 : y_0 + y_size] = 0.0
else:
    m_mid = m // 2
    n_mid = n // 2
    if node.tl:
        fill_matrix_image(node.tl, matrix_image, i, j, m_mid, n_mid)
    if node.tr:
        fill_matrix_image(node.tr, matrix_image, i, j + n_mid, m_mid, n -
↪n_mid)
    if node.bl:
        fill_matrix_image(node.bl, matrix_image, i + m_mid, j, m - m_mid,
↪n_mid)
    if node.br:
        fill_matrix_image(
            node.br, matrix_image, i + m_mid, j + n_mid, m - m_mid, n -
↪n_mid
        )

```

```

[14]: def create_matrix_image(root, shape):
    matrix_image = np.ones(shape, dtype=np.float32)

    fill_matrix_image(root, matrix_image, 0, 0, shape[0], shape[1])

    return matrix_image

```

```

[15]: def estimate_compressed_size(root):
    if root is None:
        return 0

    size = 0

    # Size of pointers to u, s, vt, tl, tr, bl, br
    pointer_size = 8 # 64-bit pointers
    num_pointers = 7
    size += num_pointers * pointer_size

    if root.is_leaf:
        size += root.u.nbytes if root.u is not None else 0
        size += root.s.nbytes if root.s is not None else 0
        size += root.vt.nbytes if root.vt is not None else 0
    else:
        size += estimate_compressed_size(root.tl)

```

```

        size += estimate_compressed_size(root.tr)
        size += estimate_compressed_size(root.bl)
        size += estimate_compressed_size(root.br)

    return size

```

```

[16]: def compress_image_demo(
    image: ImageArray, max_block_size: int, target_svd_components: int
):
    """
    Parameters
        image : ImageArray
            Image to compress.
        max_block_size : int
            Maximum block size to use for the SVD decomposition.
            (maximal rank)
        target_svd_components : int
            Number of singular values to keep in the SVD decomposition.
            (discard other singular values)
    """
    r_channel, g_channel, b_channel = extract_rgb(image)

    # Compute singular values
    r_singular_values = np.linalg.svdvals(r_channel)
    g_singular_values = np.linalg.svdvals(g_channel)
    b_singular_values = np.linalg.svdvals(b_channel)

    # Compute singular value threshold
    r_sv_threshold = get_sv_threshold(r_singular_values, target_svd_components)
    g_sv_threshold = get_sv_threshold(g_singular_values, target_svd_components)
    b_sv_threshold = get_sv_threshold(b_singular_values, target_svd_components)

    # Create hierarchical SVD representation
    root_r = create_tree(r_channel, max_block_size, r_sv_threshold)
    root_g = create_tree(g_channel, max_block_size, g_sv_threshold)
    root_b = create_tree(b_channel, max_block_size, b_sv_threshold)

    # Extract compressed matrices
    bitmap_r = root_r.matrix
    bitmap_g = root_g.matrix
    bitmap_b = root_b.matrix

    # Compressed image using the hierarchical SVD representation.
    bitmap = ImageArray(np.stack((bitmap_r, bitmap_g, bitmap_b), axis=-1))
    plot_image(
        bitmap,

```

```

        f"Compressed image (${r}=${max_block_size},\n
↳$|\\delta|=${target_svd_components})",
    )

    # Estimate the difference between the original and compressed images.
    error_r = np.linalg.norm(r_channel - bitmap_r)
    error_g = np.linalg.norm(g_channel - bitmap_g)
    error_b = np.linalg.norm(b_channel - bitmap_b)
    print(f"Error (Red channel): {error_r:.4f}")
    print(f"Error (Green channel): {error_g:.4f}")
    print(f"Error (Blue channel): {error_b:.4f}")
    error = error_r + error_g + error_b
    print(f"Total error: {error:.4f}")

    # Estimate the compression ratio.
    original_size = image.nbytes
    print(f"Original size: {original_size:_} bytes")
    compressed_size = (
        estimate_compressed_size(root_r)
        + estimate_compressed_size(root_g)
        + estimate_compressed_size(root_b)
    )
    print(f"Compressed size: {compressed_size:_} bytes")
    compression_ratio = 0 if compressed_size == 0 else original_size / \n
↳compressed_size
    print(f"Compression ratio: {compression_ratio:.4f}")

    # Image representation of the compressed matrix.
    matrix_image_r = create_matrix_image(root_r, bitmap_r.shape)
    matrix_image_g = create_matrix_image(root_g, bitmap_g.shape)
    matrix_image_b = create_matrix_image(root_b, bitmap_b.shape)

    # Image representation of the compressed matrix.
    matrix_image = ImageArray(
        np.stack((matrix_image_r, matrix_image_g, matrix_image_b), axis=-1)
    )
    plot_matrix_image(bitmap, matrix_image, "Matrix image")

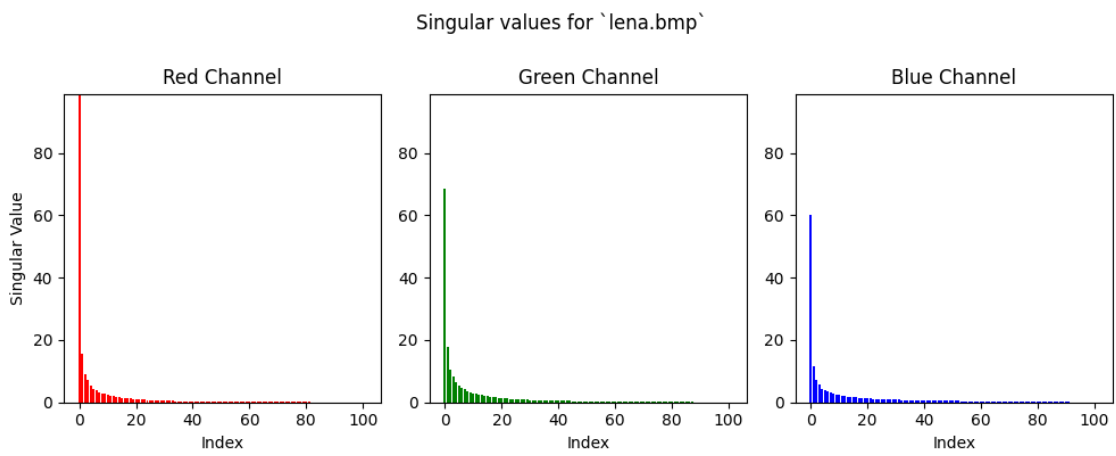
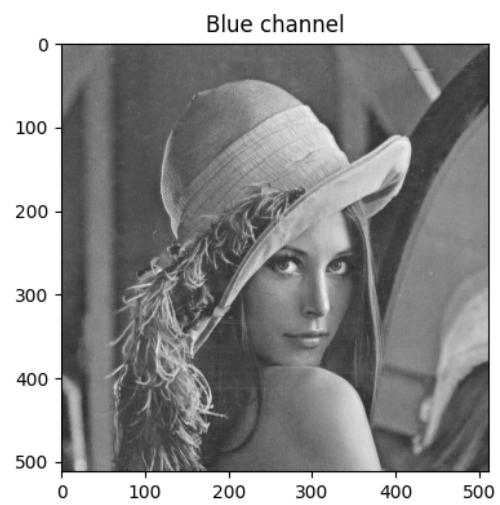
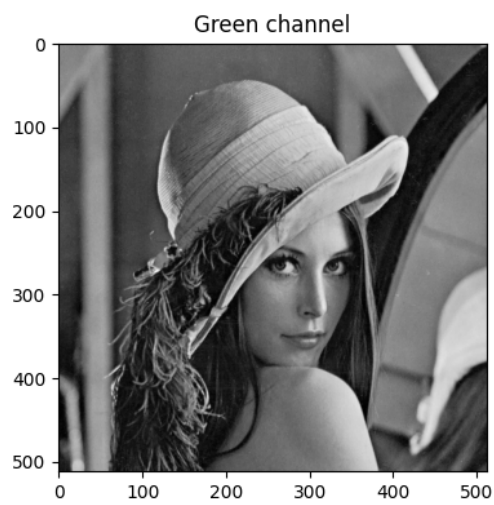
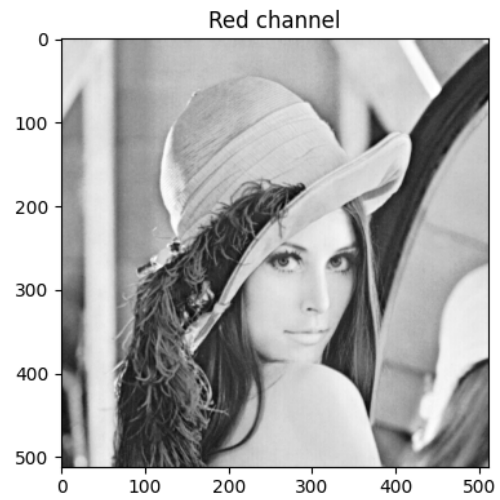
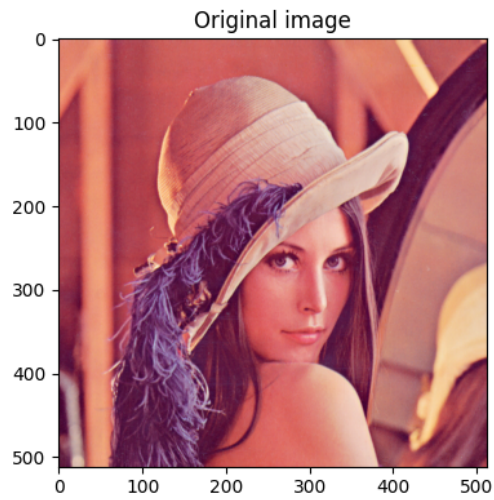
```

1.2.1 Obraz testowy Lena

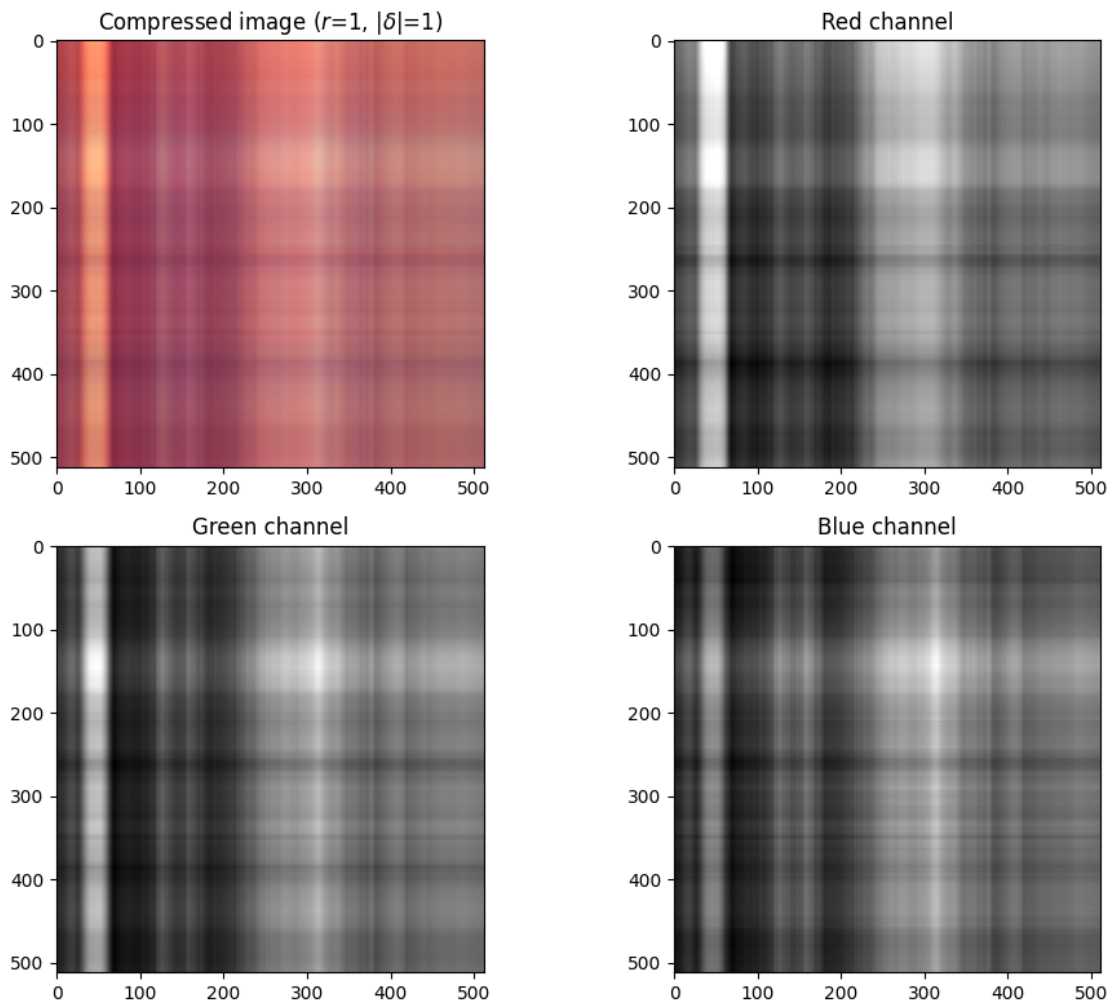
```

[17]: lena = load_image(r"data/lena.bmp")
      plot_image(lena, "Original image")
      plot_singular_values(lena, "Singular values for `lena.bmp`")

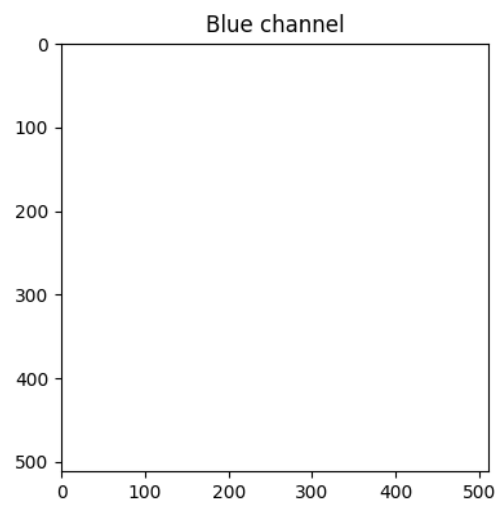
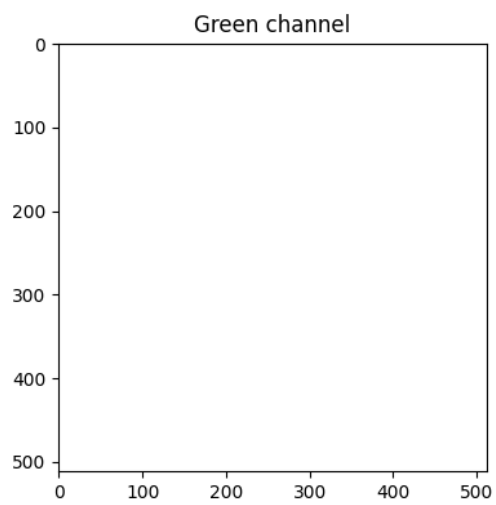
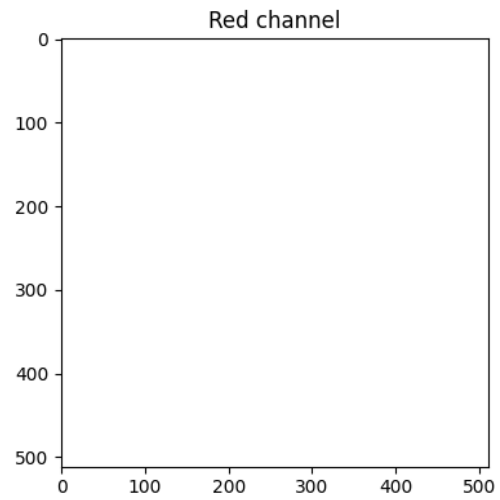
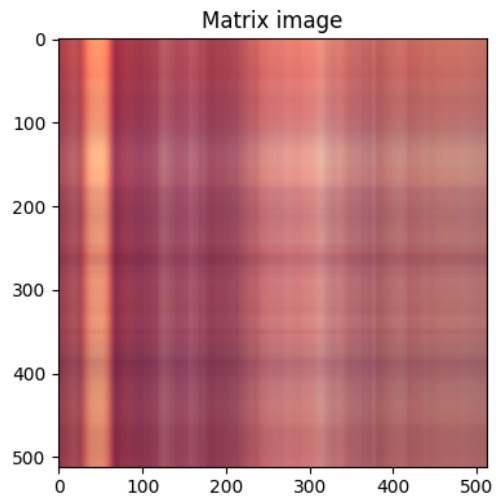
```



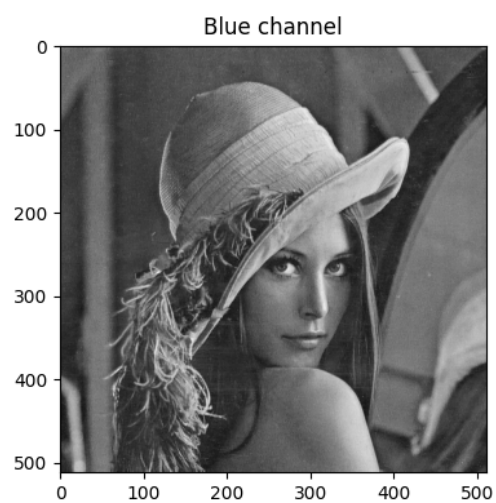
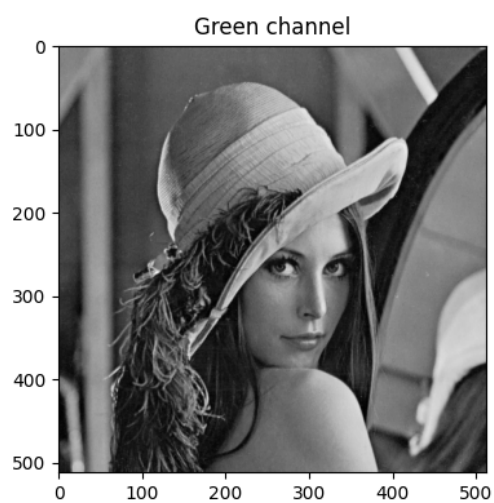
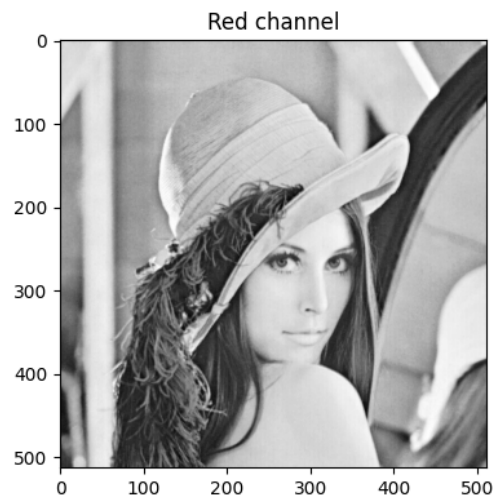
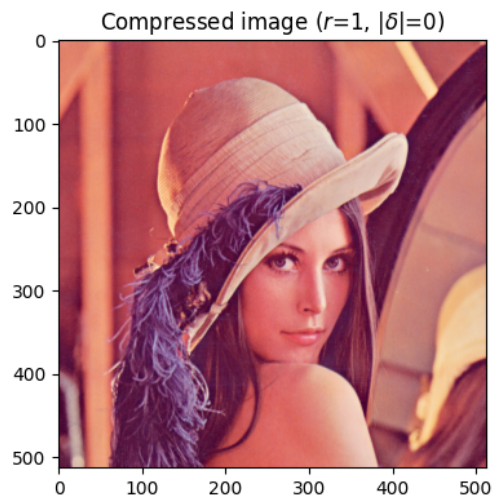
```
[18]: compress_image_demo(lena, 1, 1)
```



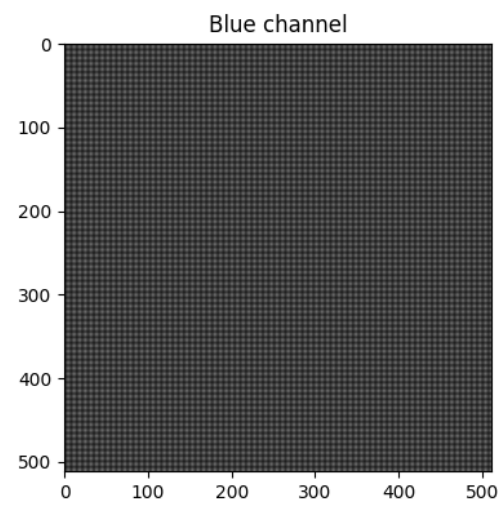
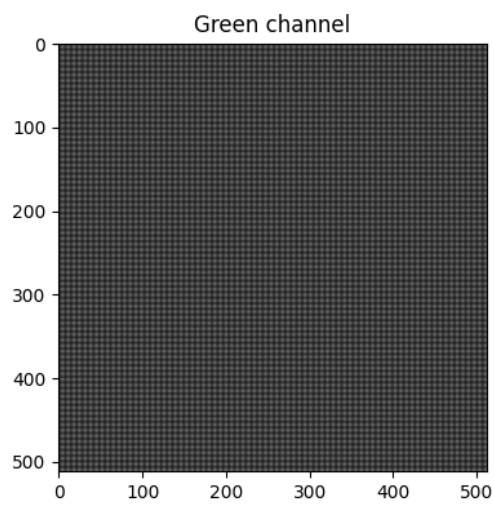
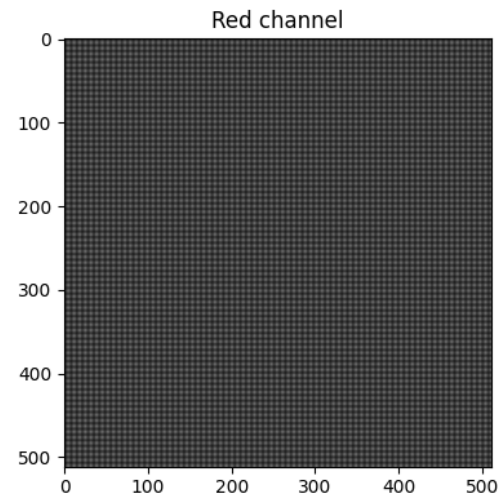
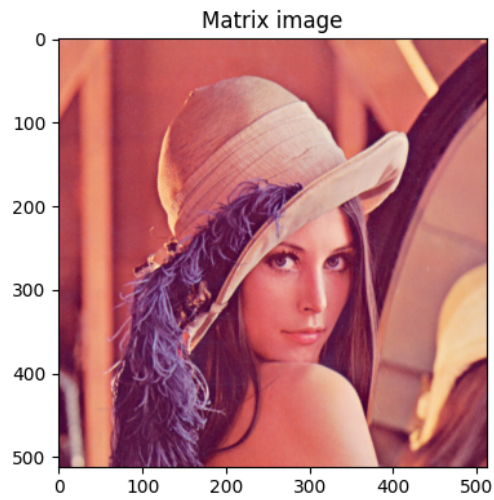
```
Error (Red channel): 83.4186
Error (Green channel): 91.6816
Error (Blue channel): 60.6110
Total error: 235.7112
Original size: 3_145_728 bytes
Compressed size: 12_468 bytes
Compression ratio: 252.3041
```



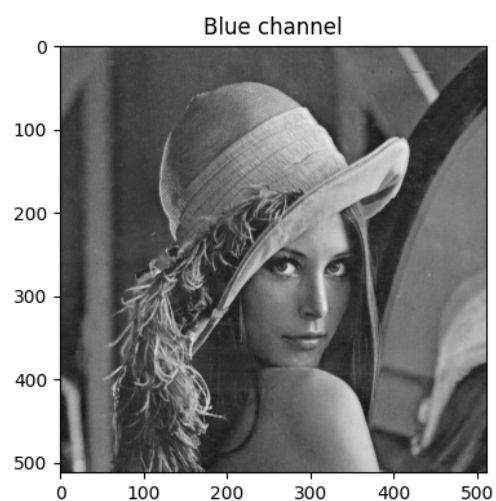
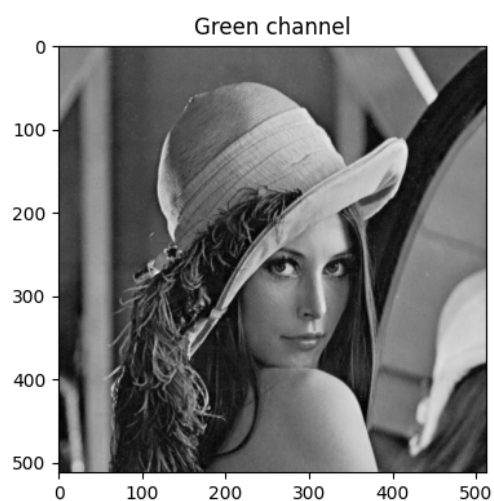
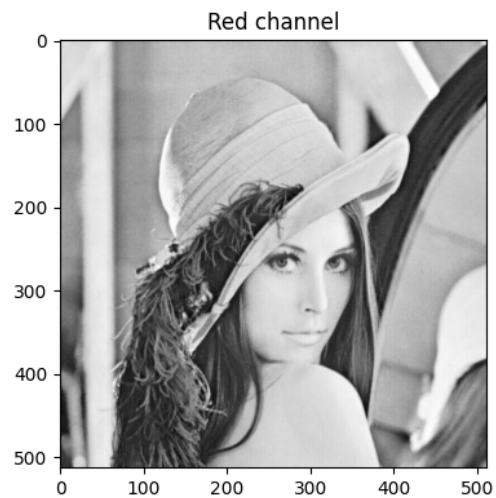
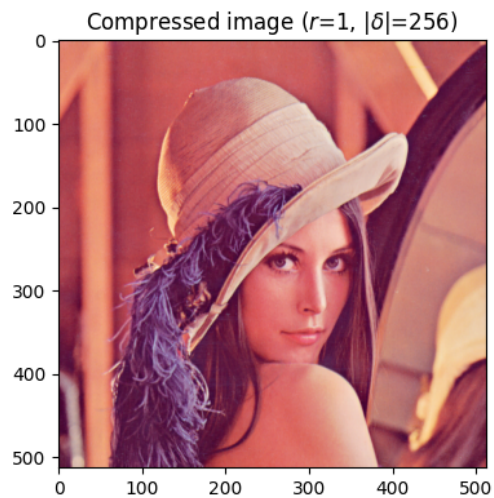
```
[19]: compress_image_demo(lena, 1, 0)
```



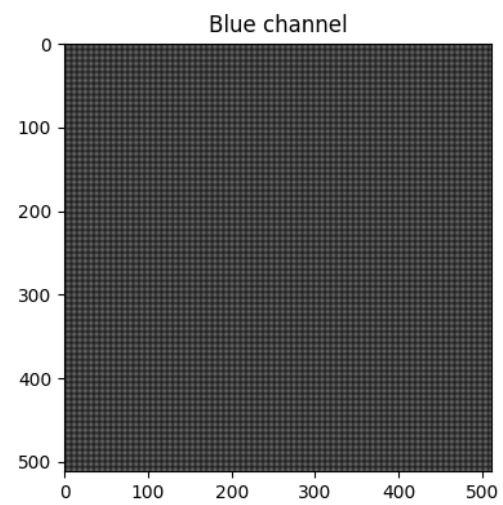
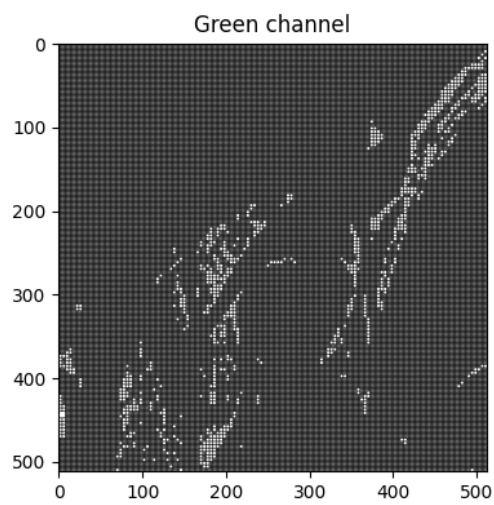
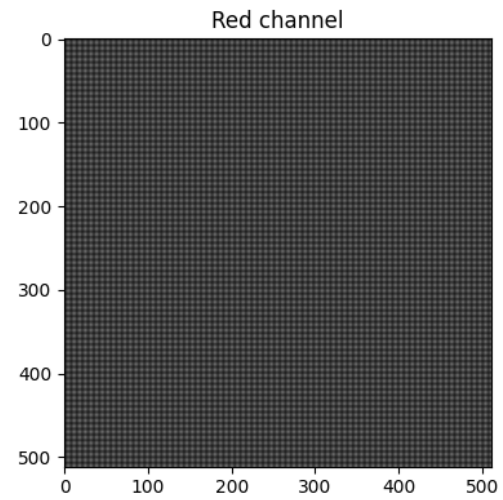
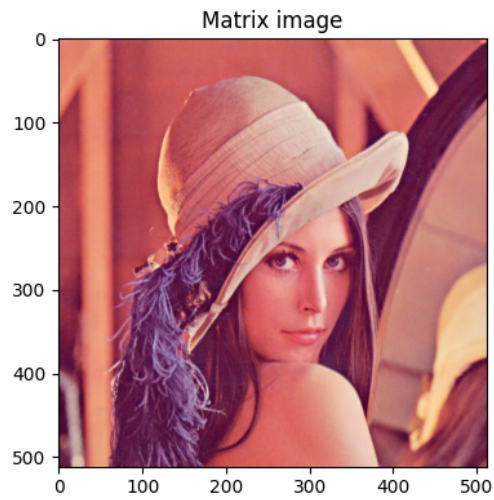
Error (Red channel): 3.3467
 Error (Green channel): 4.9650
 Error (Blue channel): 6.0188
 Total error: 14.3306
 Original size: 3_145_728 bytes
 Compressed size: 18_612_168 bytes
 Compression ratio: 0.1690



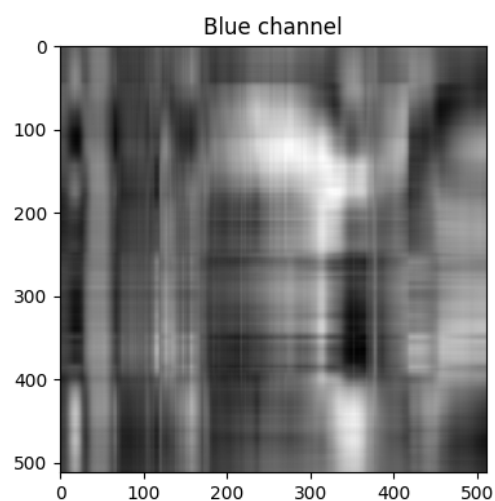
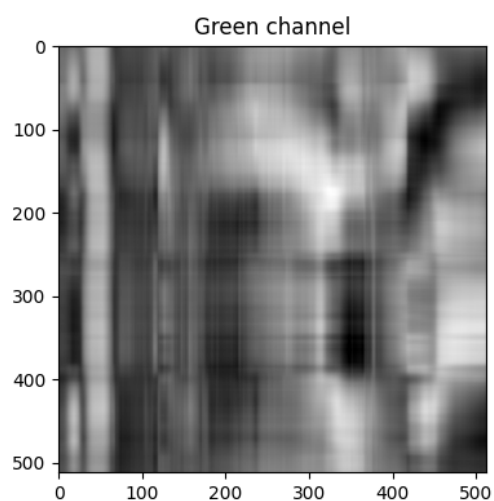
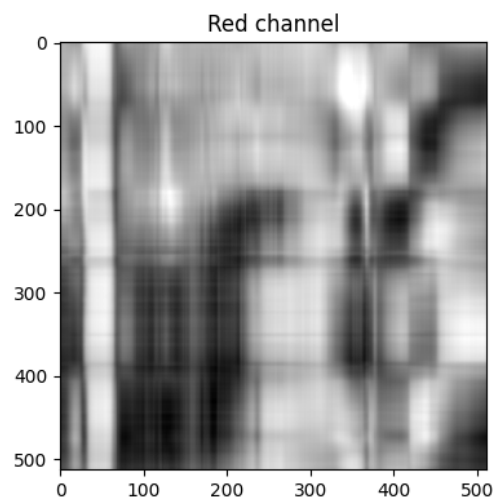
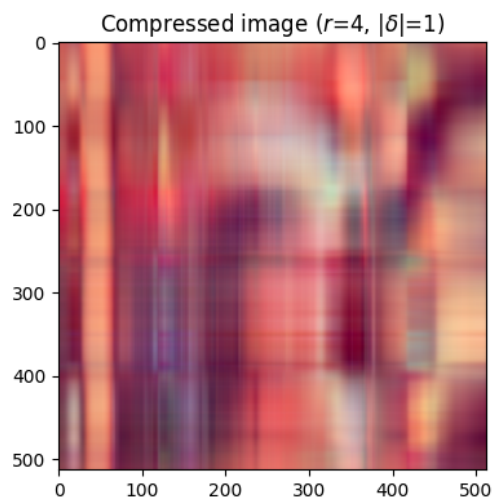
```
[20]: compress_image_demo(lena, 1, 256)
```



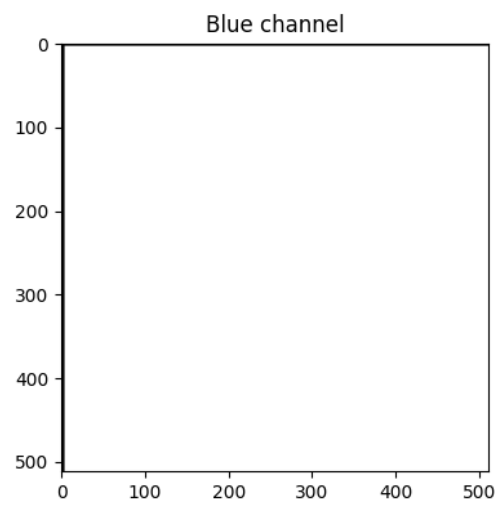
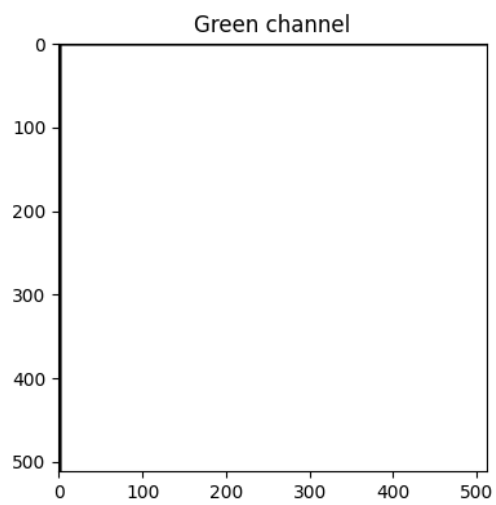
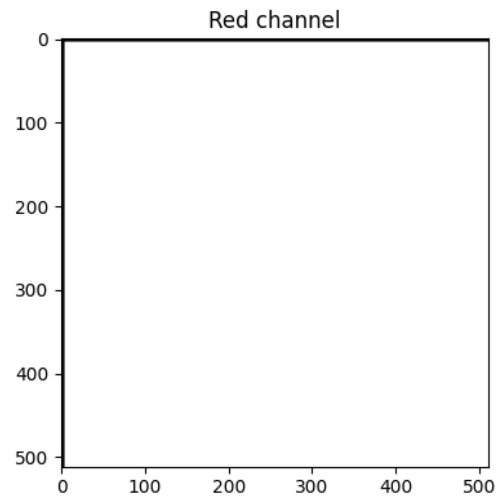
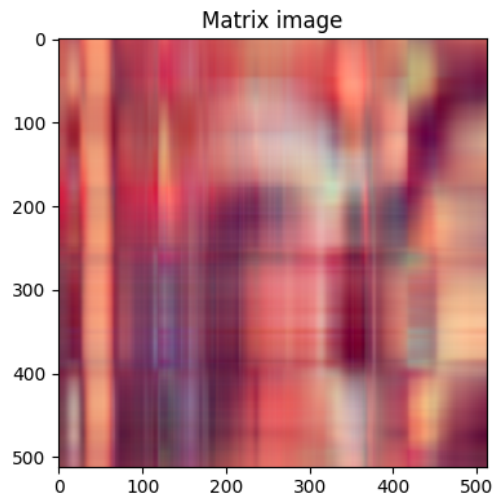
Error (Red channel): 3.3467
 Error (Green channel): 5.1305
 Error (Blue channel): 6.0188
 Total error: 14.4960
 Original size: 3_145_728 bytes
 Compressed size: 18_369_596 bytes
 Compression ratio: 0.1712



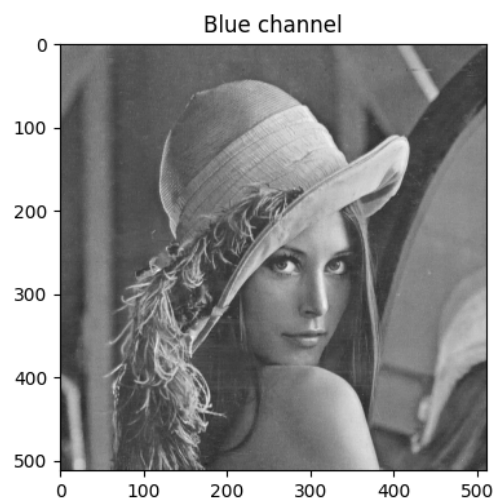
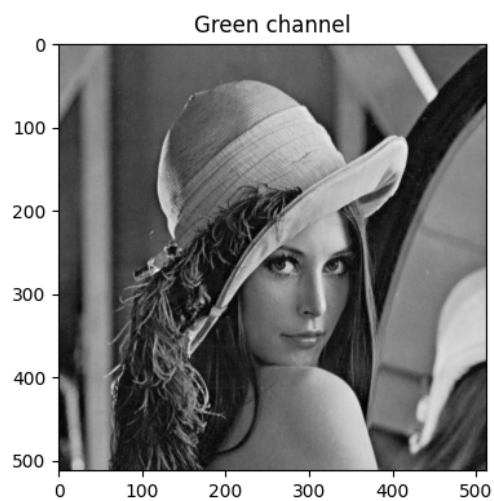
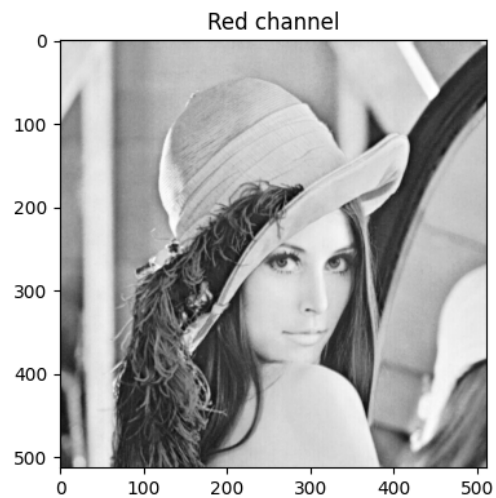
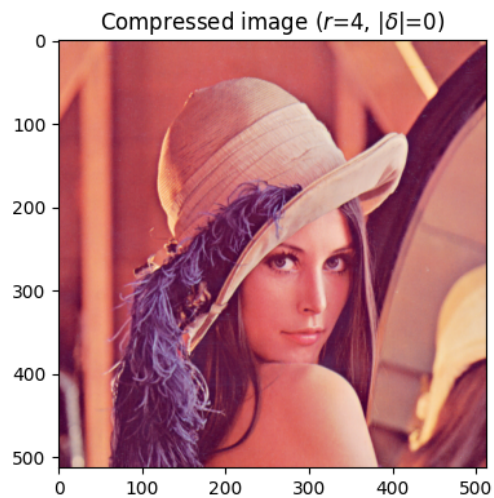
```
[21]: compress_image_demo(lena, 4, 1)
```



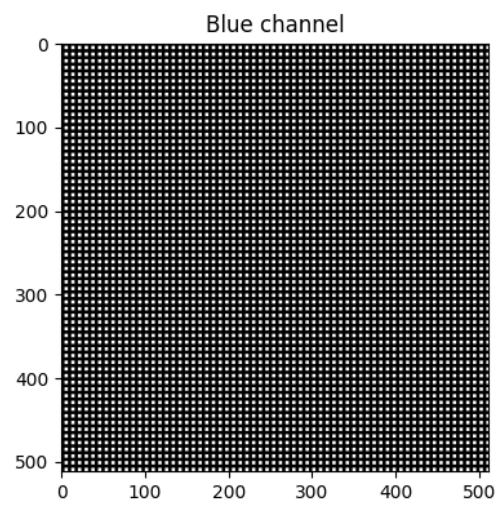
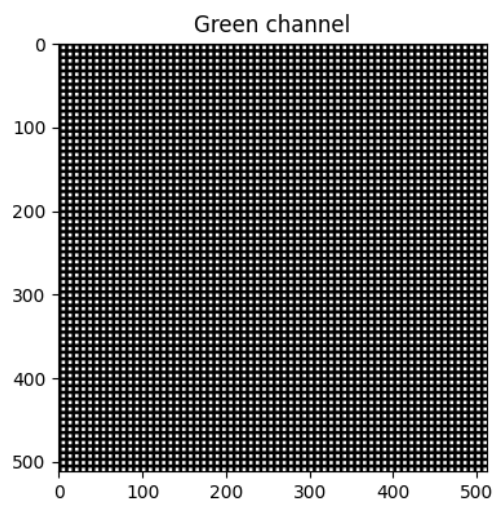
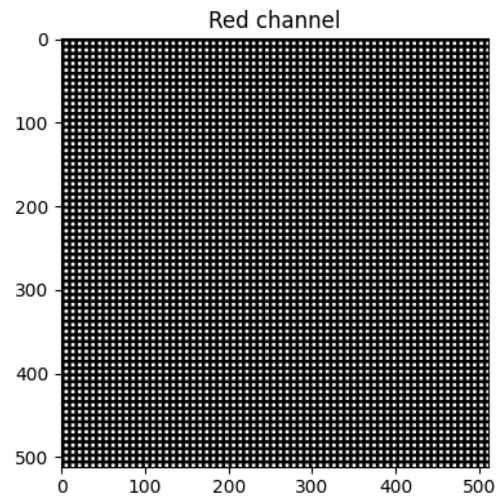
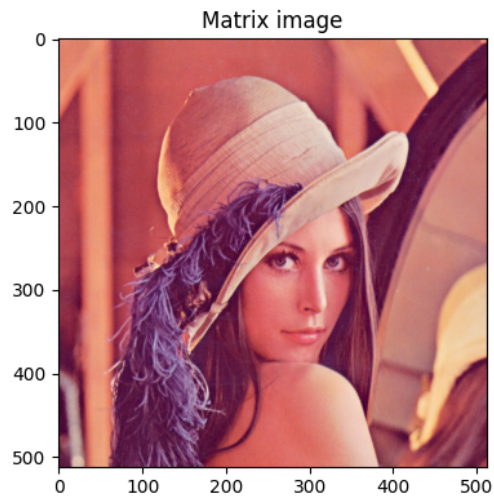
Error (Red channel): 55.7715
Error (Green channel): 64.3694
Error (Blue channel): 44.5804
Total error: 164.7213
Original size: 3_145_728 bytes
Compressed size: 49_368 bytes
Compression ratio: 63.7200



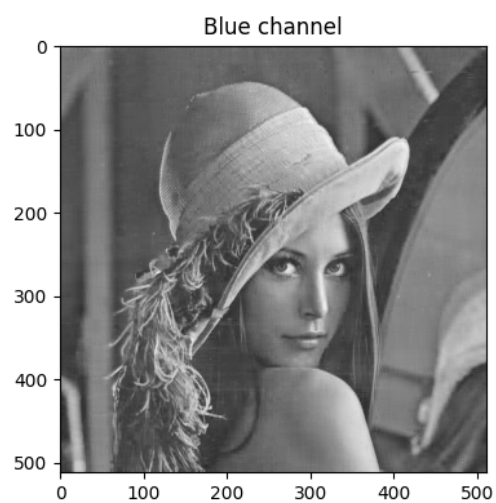
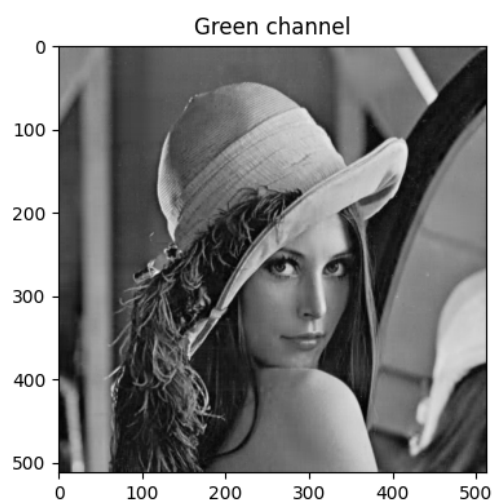
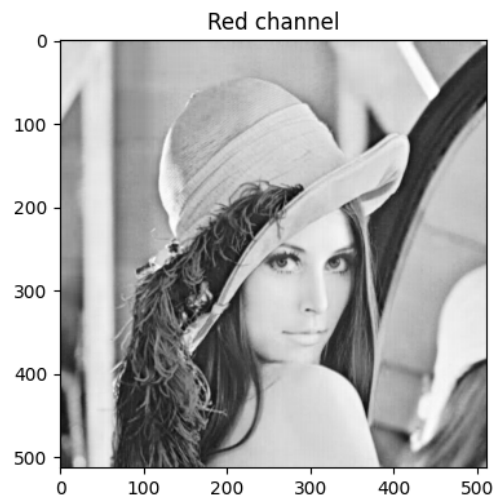
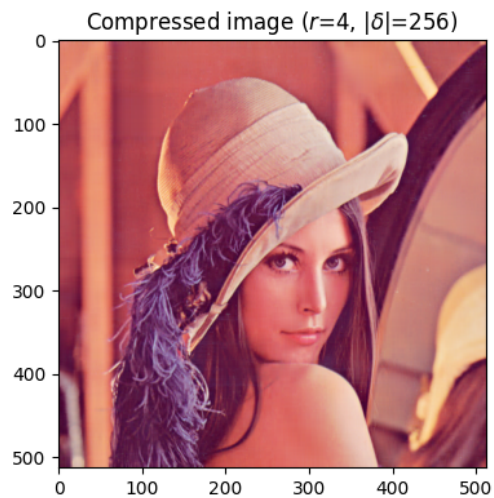
```
[22]: compress_image_demo(lena, 4, 0)
```



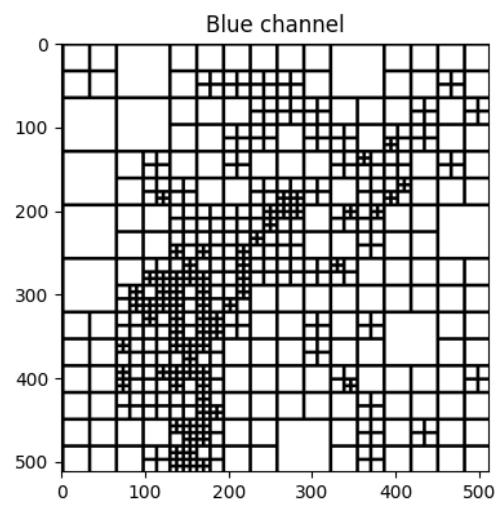
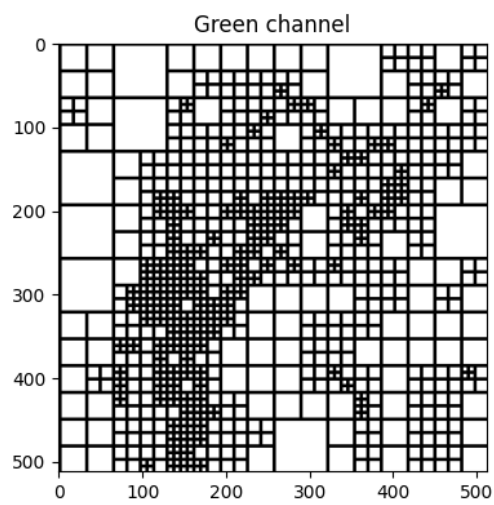
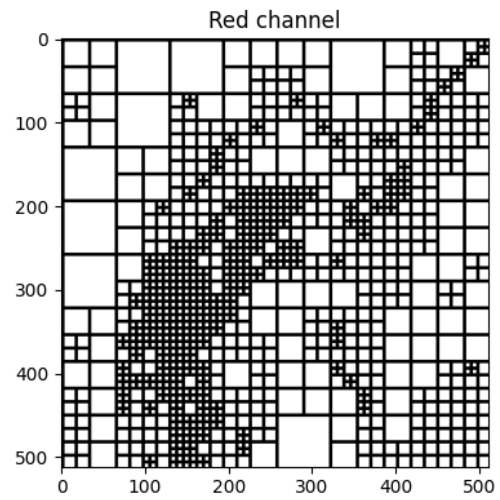
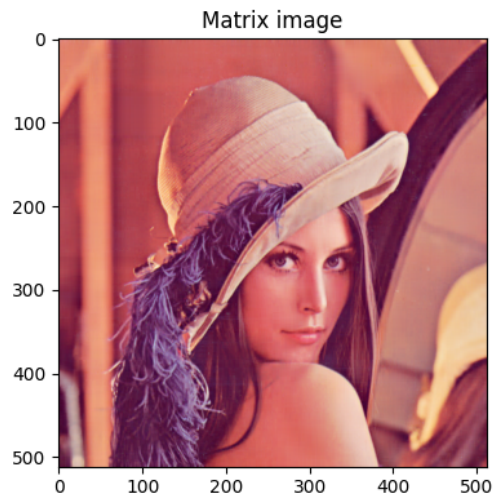
Error (Red channel): 2.3581
 Error (Green channel): 3.4577
 Error (Blue channel): 4.2416
 Total error: 10.0574
 Original size: 3_145_728 bytes
 Compressed size: 4_259_784 bytes
 Compression ratio: 0.7385



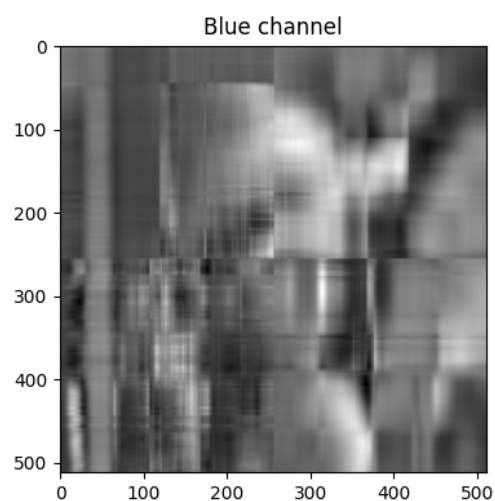
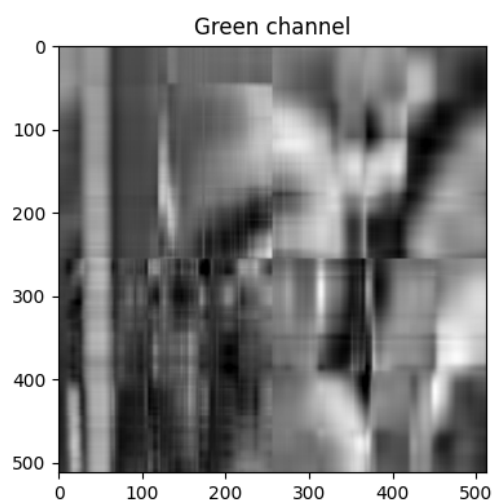
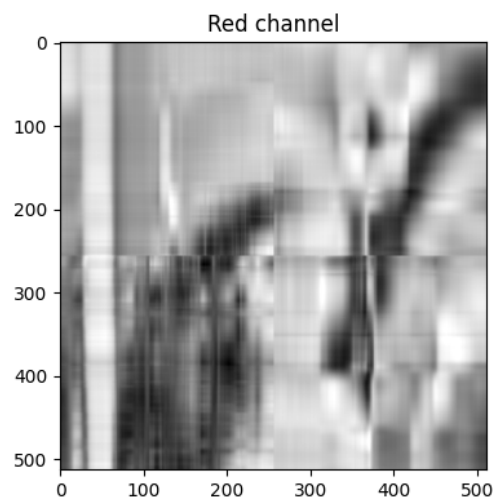
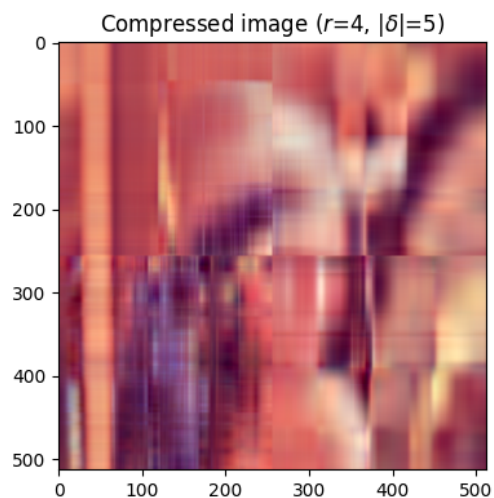
```
[23]: compress_image_demo(lena, 4, 256)
```



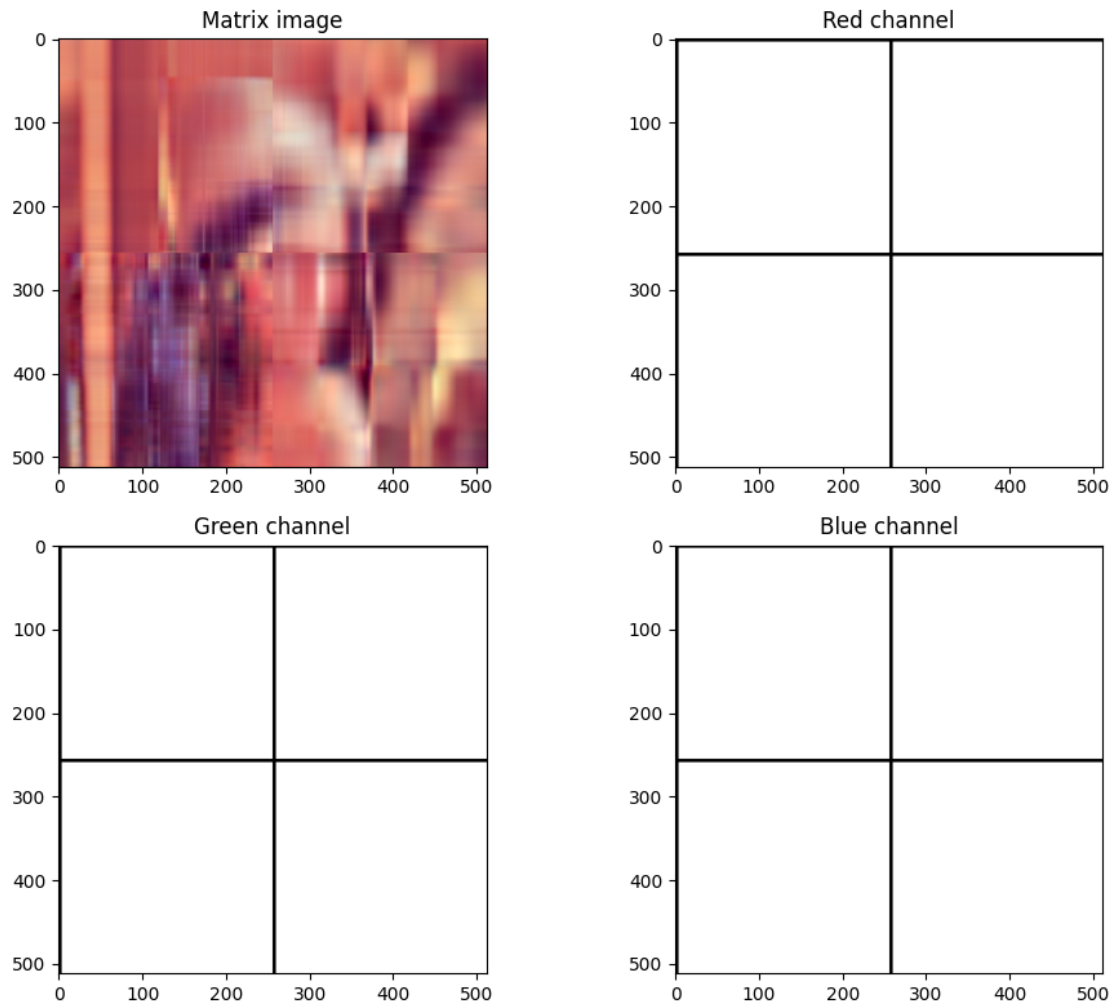
Error (Red channel): 4.5610
 Error (Green channel): 6.9108
 Error (Blue channel): 9.5082
 Total error: 20.9800
 Original size: 3_145_728 bytes
 Compressed size: 1_566_104 bytes
 Compression ratio: 2.0086



```
[24]: compress_image_demo(lena, 4, 5)
```

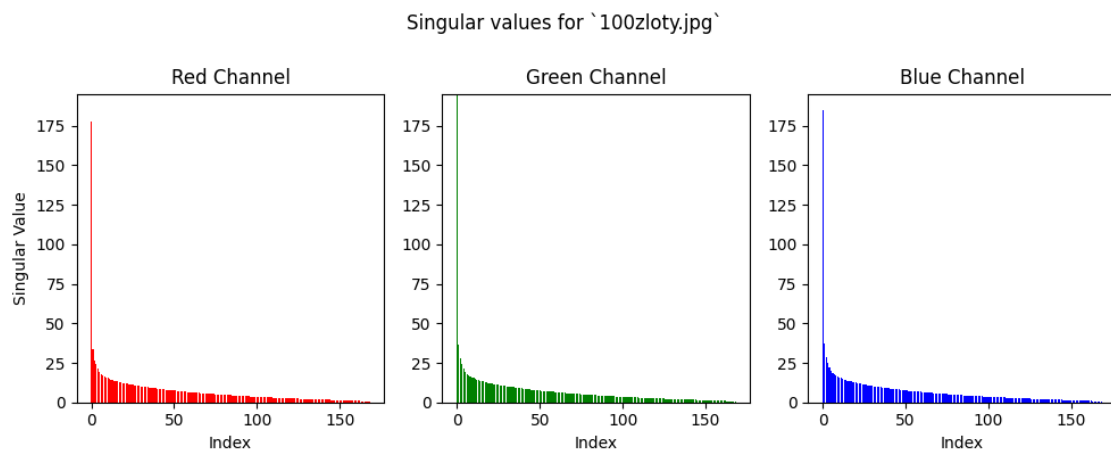
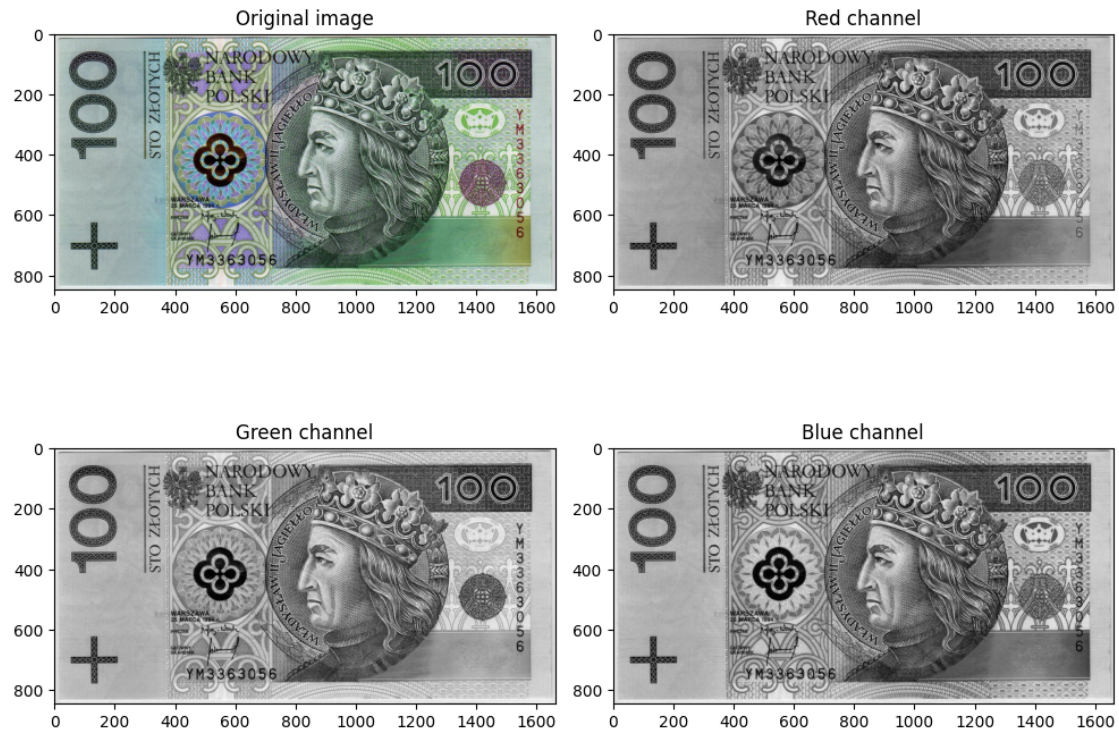


Error (Red channel): 40.7092
 Error (Green channel): 48.9141
 Error (Blue channel): 35.6340
 Total error: 125.2573
 Original size: 3_145_728 bytes
 Compressed size: 99_336 bytes
 Compression ratio: 31.6676

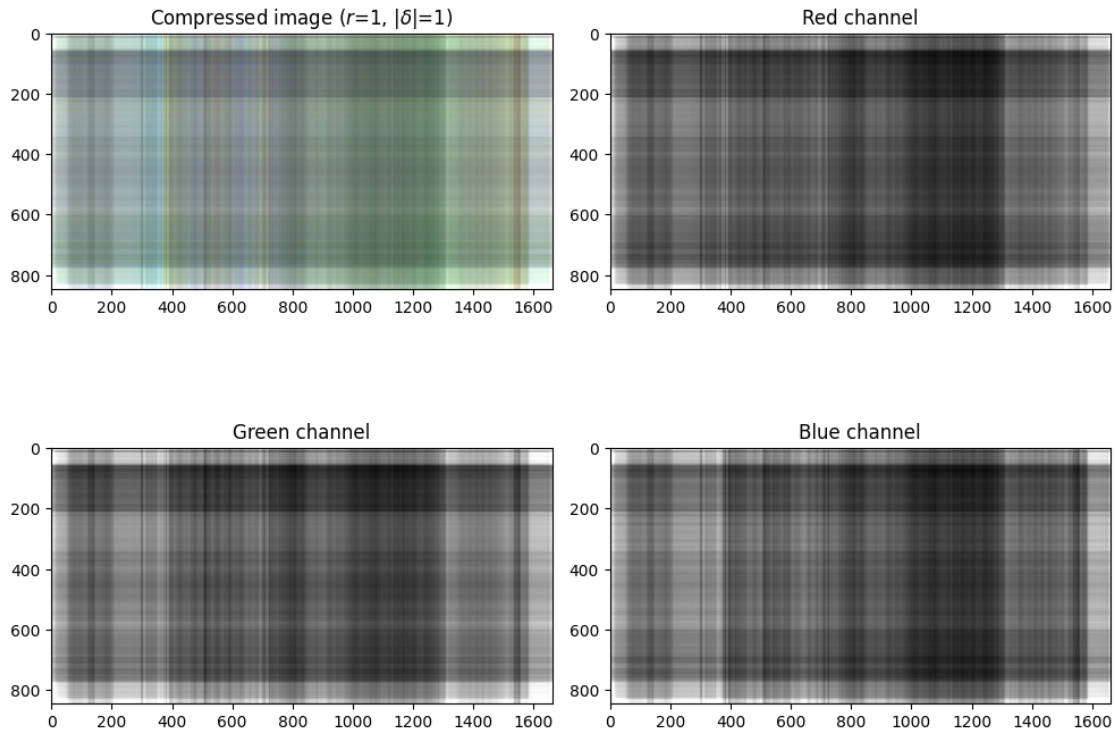


1.2.2 Inny obraz testowy

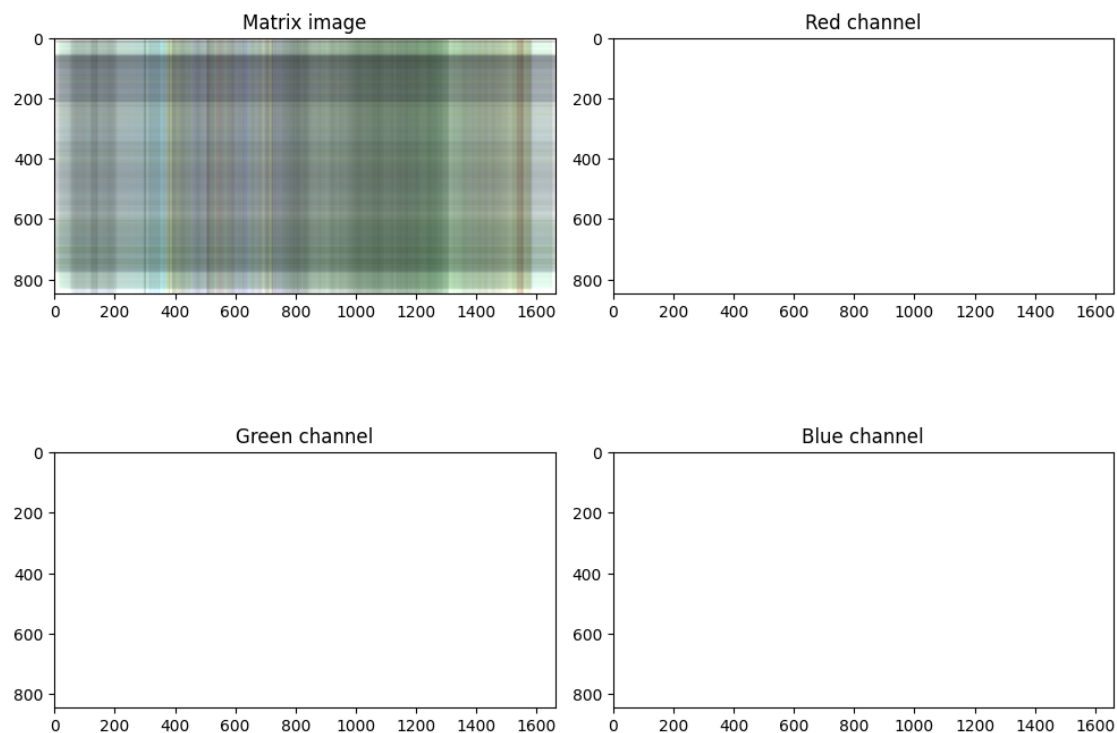
```
[25]: image = load_image(r"data/100zloty.jpg")  
      plot_image(image, "Original image")  
      plot_singular_values(image, "Singular values for `100zloty.jpg`")
```



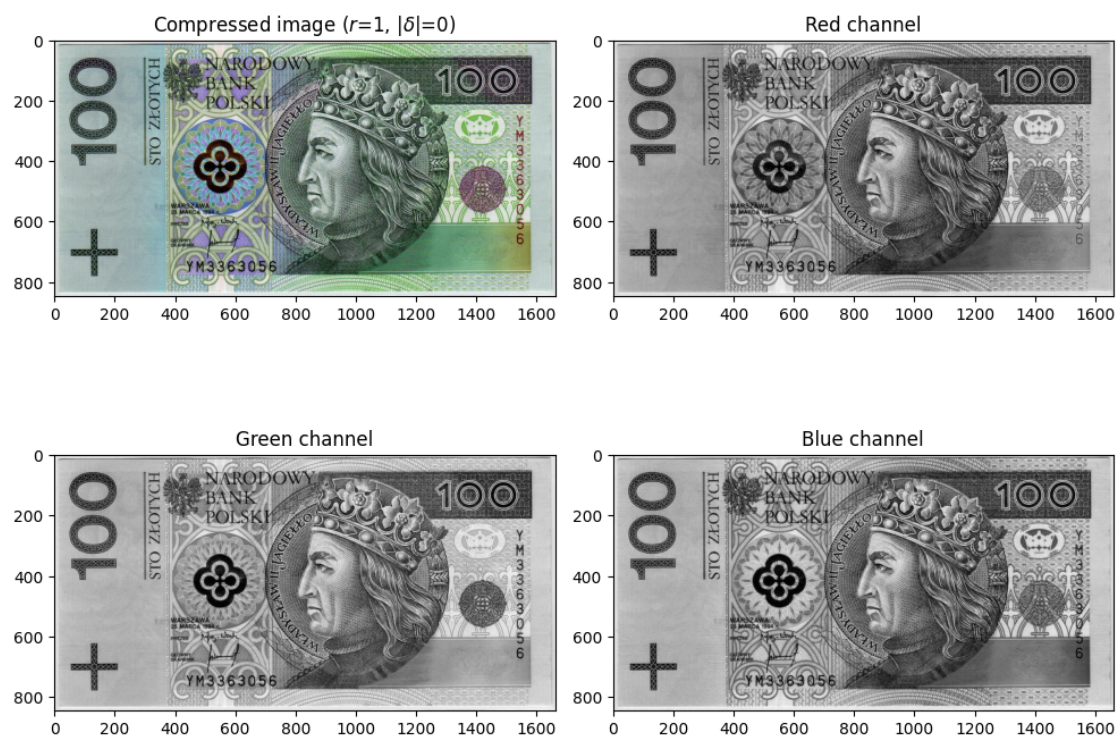
```
[26]: compress_image_demo(image, 1, 1)
```



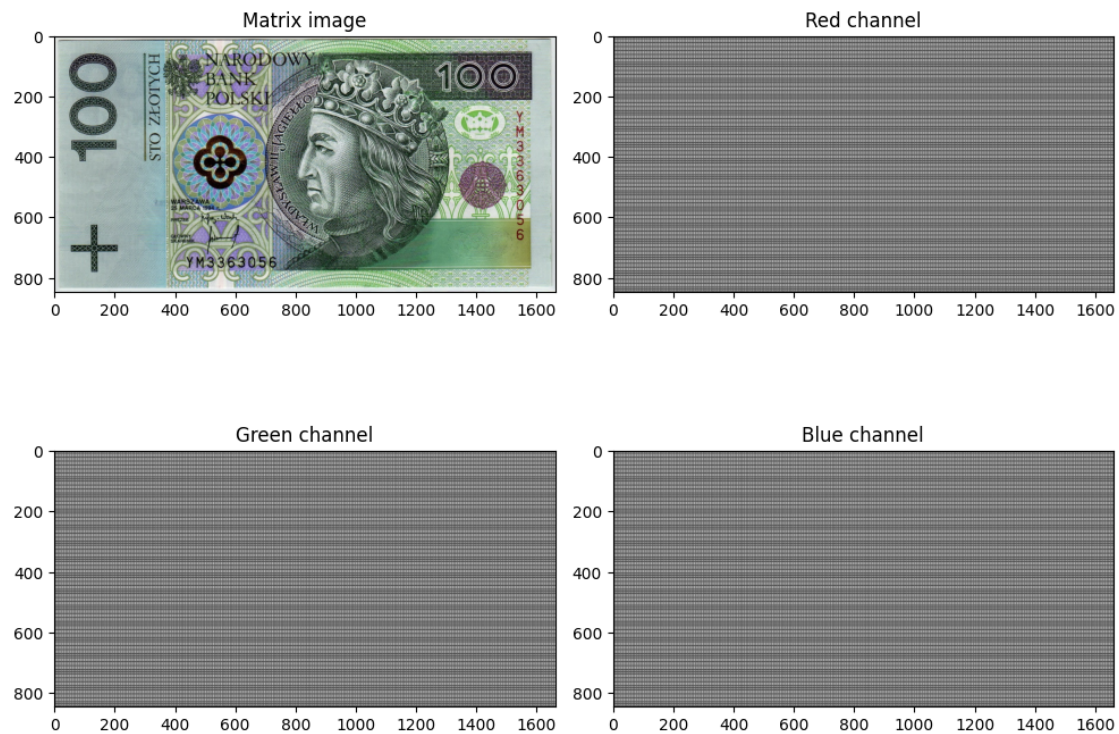
Error (Red channel): 263.2455
Error (Green channel): 269.8534
Error (Blue channel): 272.0210
Total error: 805.1199
Original size: 16_882_404 bytes
Compressed size: 30_276 bytes
Compression ratio: 557.6167



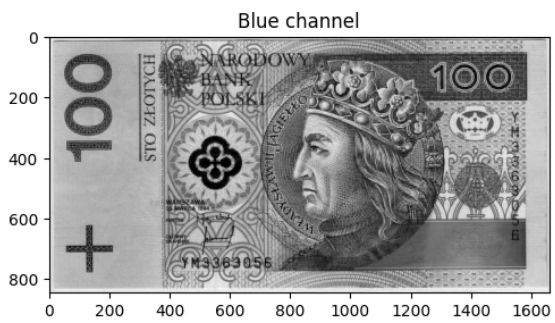
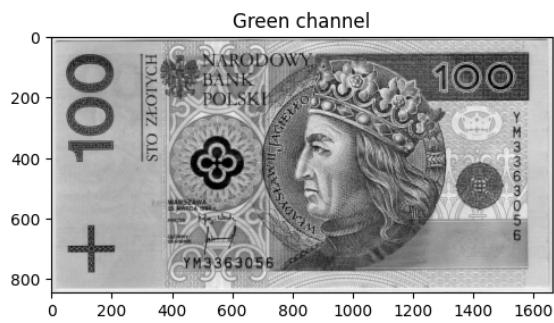
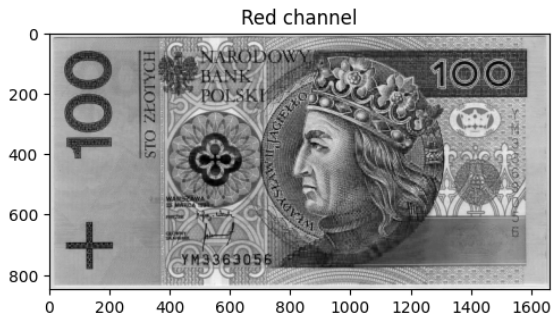
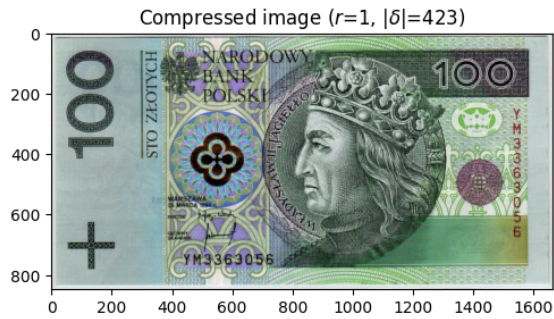
[27]: `compress_image_demo(image, 1, 0)`



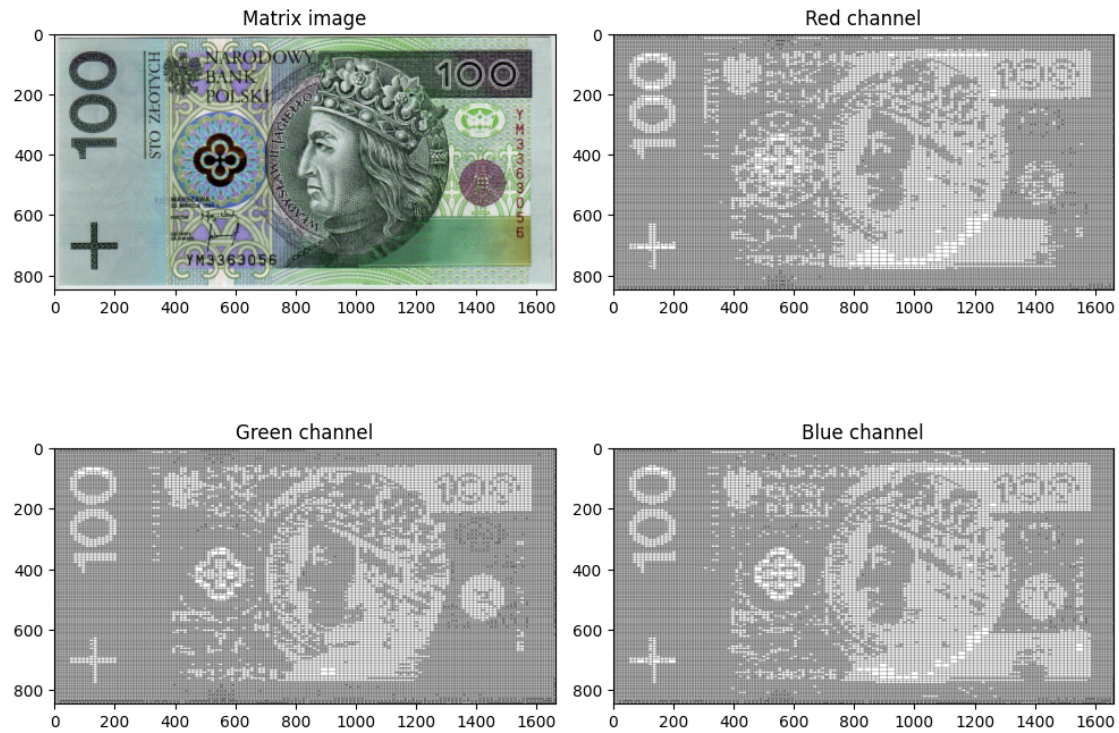
Error (Red channel): 106.5618
Error (Green channel): 107.6806
Error (Blue channel): 107.1714
Total error: 321.4138
Original size: 16_882_404 bytes
Compressed size: 40_034_988 bytes
Compression ratio: 0.4217



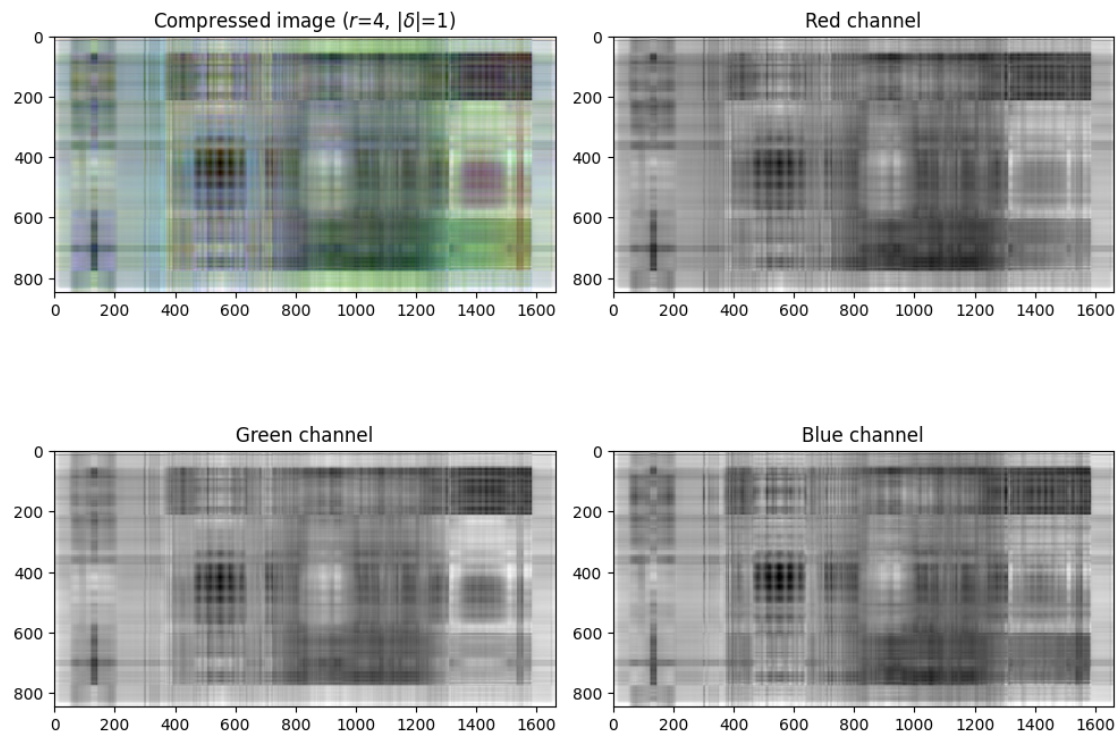
```
[28]: compress_image_demo(image, 1, 423)
```



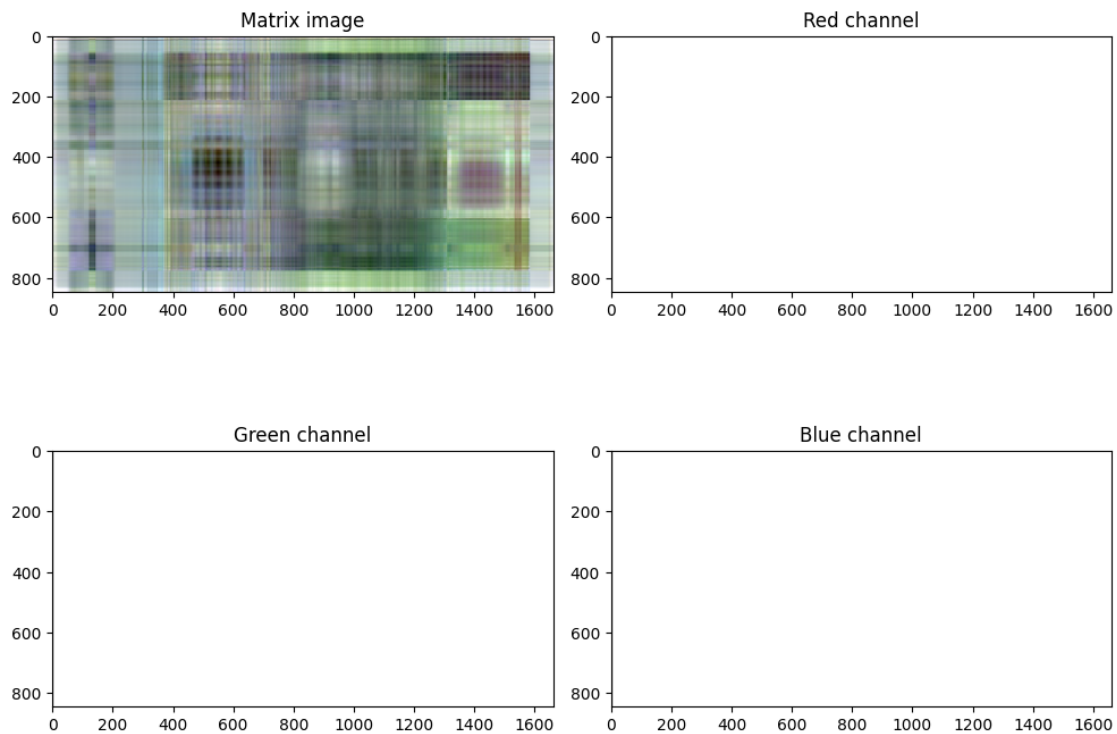
Error (Red channel): 148.6492
 Error (Green channel): 145.7432
 Error (Blue channel): 148.3973
 Total error: 442.7897
 Original size: 16_882_404 bytes
 Compressed size: 18_560_548 bytes
 Compression ratio: 0.9096



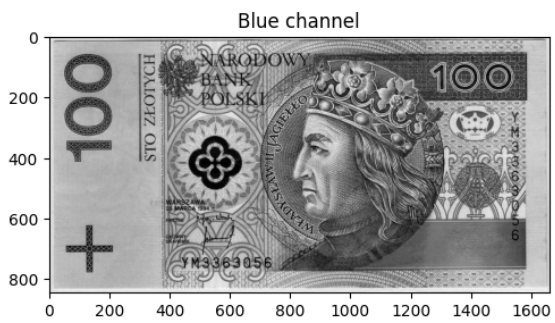
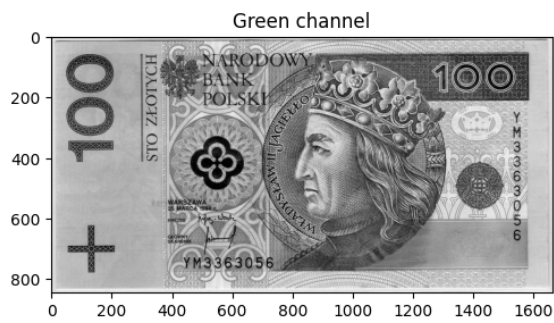
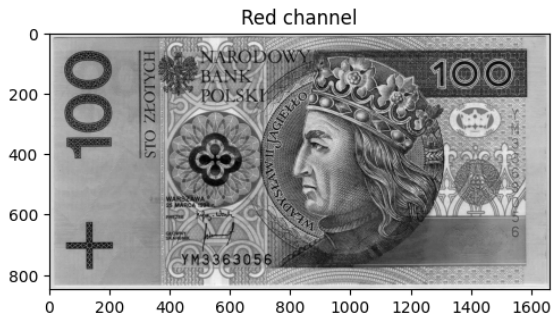
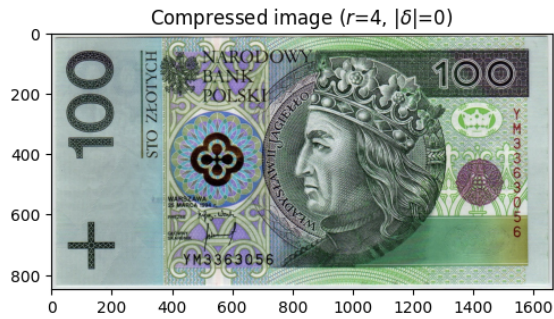
```
[29]: compress_image_demo(image, 4, 1)
```



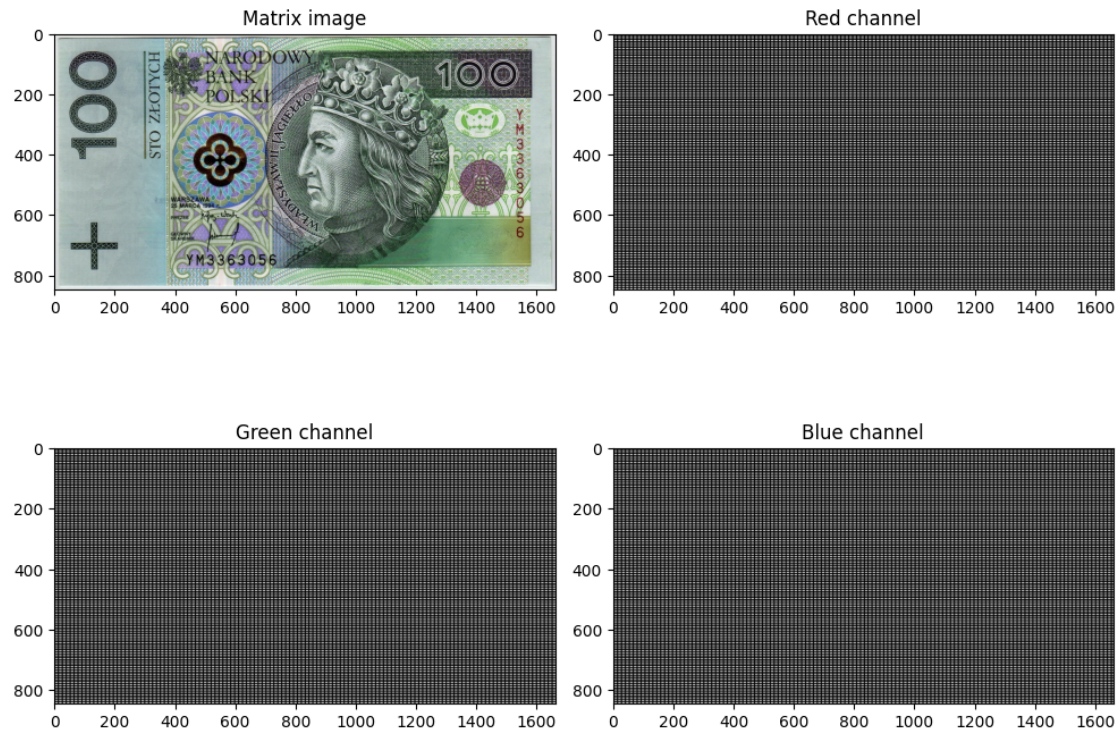
Error (Red channel): 238.8192
Error (Green channel): 242.8009
Error (Blue channel): 247.9315
Total error: 729.5516
Original size: 16_882_404 bytes
Compressed size: 120_600 bytes
Compression ratio: 139.9868



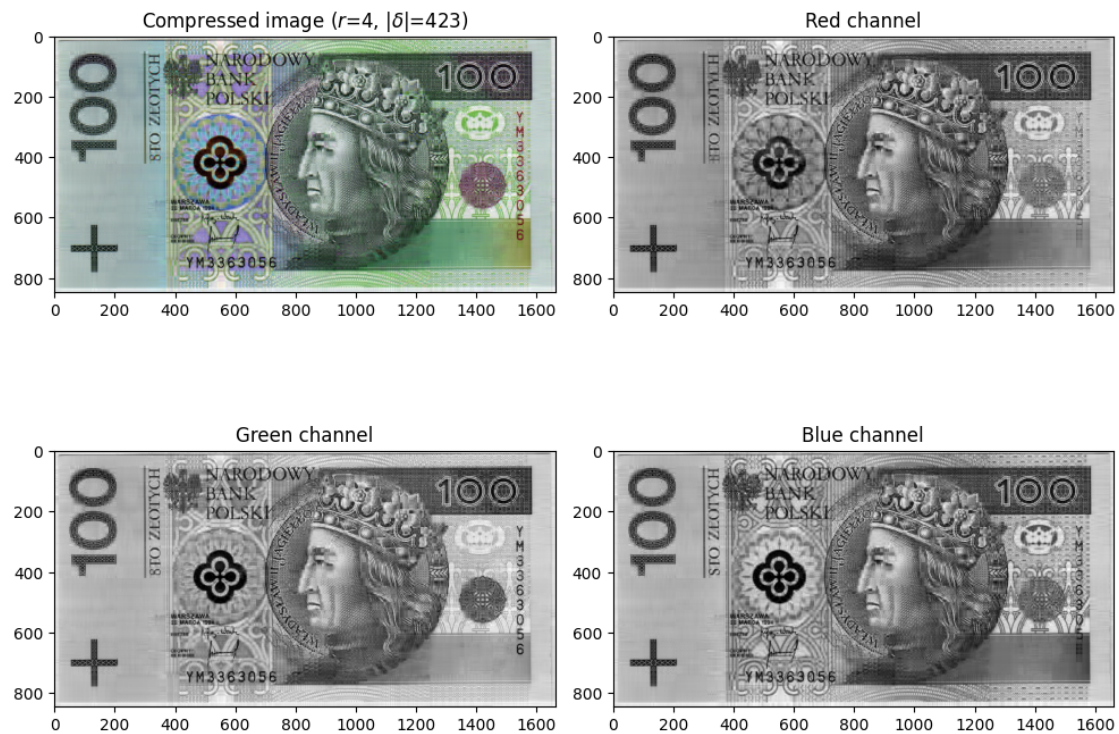
```
[30]: compress_image_demo(image, 4, 0)
```



Error (Red channel): 47.5494
 Error (Green channel): 48.2160
 Error (Blue channel): 47.9301
 Total error: 143.6956
 Original size: 16_882_404 bytes
 Compressed size: 19_865_544 bytes
 Compression ratio: 0.8498



```
[31]: compress_image_demo(image, 4, 423)
```



Error (Red channel): 149.3392
Error (Green channel): 147.6107
Error (Blue channel): 148.7794
Total error: 445.7293
Original size: 16_882_404 bytes
Compressed size: 2_621_816 bytes
Compression ratio: 6.4392

